

Министерство Образования Российской Федерации
УрФУ
Образовательных информационных технологий

Лабораторная работа №5
Тема: «Форматирование и линтинг проекта. Сборка образа проекта»

Выполнил:
Нарсеев А.В.

Екатеринбург, 2025

Лабораторная работа №5: Форматирование и линтинг проекта. Сборка образа проекта

Цель: познакомиться со способами поддержки качества кода и сборки образа приложения. Лабораторная работа выполняется на основе ЛР4, в репозитории после выполнения добавляем тэг lab_5.

Применение линтеров и формatters при локальной разработке

Установим в проект pre-commit, pylint, black и isort.

В корне проекта настраиваем .pre-commit-config.yaml:

```
! .pre-commit-config.yaml
1  fail_fast: false
2
3  repos:
4    - repo: local
5      hooks:
6        - id: system
7          name: Black
8          entry: black LR5/app
9          pass_filenames: false
10         language: system
11    - repo: local
12      hooks:
13        - id: isort
14          name: isort
15          entry: isort LR5/app
16          pass_filenames: false
17          language: system
18    - repo: local
19      hooks:
20        - id: system
21          name: pylint
22          entry: bash -c 'cd LR5 && pylint app'
23          pass_filenames: false
24          language: system
25
```

.pylintrc:

```
≡ .pylintrc
1  [BASIC]
2  init-hook='import sys, os; sys.path.append(os.path.join(os.path.d
3  disable =
4      missing-function-docstring,
5      missing-module-docstring,
6      missing-class-docstring,
7      too-few-public-methods,
8      not-callable,
9      attribute-defined-outside-init,
10     ungrouped-imports,
11     wrong-import-order,
12     duplicate-code,
13     unused-import,
14     too-many-arguments,
15     unused-argument,
16     import-outside-toplevel,
17     too-many-instance-attributes,
18     import-error
19
20     ignore-paths =
21         deploy,
22         temp,
23         build,
24         tests,
25         migrations
26
27     extension-pkg-whitelist=
28         pydantic,
29         orjson
30
31     generated-members =
32         [pydantic.PostgresDsn.build]
```

Black и isort в pyproject.toml:

```
5
6 [tool.isort]
7 profile = "black"
8 known_first_party = ["app"]
9 skip = []
10
11 [tool.black]
12 line-length = 88
13 target-version = ['py311']
14 exclude = '''
15 /(
16 | \.git
17 | \.hg
18 | \.mypy_cache
19 | \.tox
20 | \.venv
21 | _build
22 | buck-out
23 | build
24 | docker
25 | config
26 | temp
27 | migrations
28 )/
29 '''
```

Запускаем pre-commit run --all-files:

```
● (venv) Mac:LR5 narseev$ pre-commit run --all-files
Black.....Passed
isort.....Passed
pylint.....Passed
○ (venv) Mac:LR5 narseev$
```

Сборка образа проекта

Для сбора образа проекта нам необходимо создать Dockerfile

```
➤ Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 # Установка системных зависимостей для PostgreSQL и netcat
6 RUN apt-get update && apt-get install -y \
7     postgresql-client \
8     netcat-openbsd \
9     gcc \
10    python3-dev \
11    libpq-dev \
12    && rm -rf /var/lib/apt/lists/*
13
14 # Установка зависимостей
15 COPY requirements.txt .
16 RUN pip install --no-cache-dir -r requirements.txt
17
18 # Копирование приложения
19 COPY . .
20
21 # Настройка миграций
22 RUN chmod +x entrypoint.sh
23
24 ENTRYPOINT ["./entrypoint.sh"]
25
```

Создаем entrypoint.sh для запуска проекта

```
$ entrypoint.sh

1  #!/bin/bash
2
3  # Ожидание готовности базы данных
4  while ! nc -z $DB_HOST $DB_PORT; do sleep 0.1
5  done
6
7  # Применение миграций
8  alembic upgrade head
9
10 # Запуск приложения
11 exec "$@"
12
```

Добавляем в docker-compose.yml проект и указываем, что он зависит от базы данных

```
20
21  > Run Service
22  app:
23    build: .
24    container_name: lr5_app
25    ports:
26      - "8000:8000"
27    env_file:
28      - .env
29    environment:
30      - DB_HOST=${DB_HOST:-postgres}
31      - DB_PORT=${DB_PORT:-5432}
32      - DATABASE_URL=${DATABASE_URL_DOCKER:-postgresql+psycopg2://postgres:postgres@postgres:5432/lr5}
33      - POSTGRES_USER=${POSTGRES_USER:-postgres}
34      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD:-pass}
35      - POSTGRES_DB=${POSTGRES_DB:-labdb}
36    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
37    depends_on:
38      postgres:
39        condition: service_healthy
40    networks:
41      - lr5_network
```

Затем запускаем проект с помощью docker compose up --build и проверяем работу на порту 8000

```
narseev@Mac LR5 % docker-compose up --build -d
[+] Building 1.7s (14/14) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 549B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 613B 0.0s
=> [internal] load metadata for docker.io/library/python:3.11-slim 1.3s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 230B 0.0s
=> [1/7] FROM docker.io/library/python:3.11-slim@sha256:8ef21a26e7c342e978a68cf2d6b076278859 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:8ef21a26e7c342e978a68cf2d6b076278859 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 3.80kB 0.0s
=> CACHED [2/7] WORKDIR /app 0.0s
=> CACHED [3/7] RUN apt-get update && apt-get install -y postgresql-client netcat-op 0.0s
=> CACHED [4/7] COPY requirements.txt . 0.0s
=> CACHED [5/7] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [6/7] COPY . 0.0s
=> [7/7] RUN chmod +x entrypoint.sh 0.1s
=> exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:5f4781966942a311e57281ced3edd7e0ac00bef8b59b5e8f560f2e3a13b3 0.0s
=> => exporting config sha256:887cf2d579354e882aab983c72ca69e33a2c53ce47fa05378751acac0ffc9f 0.0s
=> => exporting attestation manifest sha256:ae9f0b20cb04b9057b1a83ac230b493a27e437bdd1f9d37e 0.0s
=> => exporting manifest list sha256:aa318f8c42b2ca084c096cb2210a8fa6610e25cd0b87df6b9b2d10b 0.0s
=> => naming to docker.io/library/lr5-app:latest 0.0s
=> => unpacking to docker.io/library/lr5-app:latest 0.0s
=> resolving provenance for metadata file 0.0s
[+] Running 4/4
✔ lr5-app Built 0.0s
✔ Network lr5_lr5_network Created 0.0s
✔ Container lr5_postgres Healthy 5.7s
✔ Container lr5_app Started 5.7s
narseev@Mac LR5 %
```

(бд перед этим заполнил тестовыми данными через fill_full_db.py)

```
Container 'lrs_app' started
narseev@Mac LR5 % curl http://localhost:8000/users
{"total":3,"users":[{"id":"12aba8e9-aecc-4bf1-ab0e-d2b78c6ab9bf","username":"alice","email":"alice@example.com","description":"Активный покупатель","created_at":"2025-11-18T21:58:15.878882","updated_at":"2025-11-18T21:58:15.878883"}, {"id":"c6d62096-1bfe-41a4-8c2a-0481e9c187be","username":"bob","email":"bob@example.com","description":"Любит технику","created_at":"2025-11-18T21:58:15.878888","updated_at":"2025-11-18T21:58:15.878888"}, {"id":"8b330025-c6f9-4724-a26c-f5dcf730ff2a","username":"carol","email":"carol@example.com","description":"Тестовый пользователь","created_at":"2025-11-18T21:58:15.878891","updated_at":"2025-11-18T21:58:15.878891"}]}
```

Вопросы

1. Что такое Docker-контейнер и чем он отличается от виртуальной машины?
Docker-контейнер - изолированная среда, которая запускает приложение вместе со всеми зависимостями, используя ядро хостовой операционной системы.
Виртуальная машина - полноценная ОС со своим ядром, поэтому тяжелее и запускается медленнее.
Контейнеры быстрее, компактнее и ориентированы на запуск приложений, а VM - на полные изолированные окружения
2. Как работает кеширование слоев в Docker и почему это важно?
Docker кеширует каждый слой образа, созданный на основе инструкций в Dockerfile, и при повторной сборке использует ранее сохранённые слои, если соответствующие команды и их контекст не изменились. Это значительно ускоряет сборку и уменьшает потребление ресурсов
3. Что означает инструкция depends_on в docker-compose?
depends_on задает порядок запуска сервисов. Зависимый сервис запускается после указанных
4. Почему миграции БД выполняются в entrypoint.sh, а не во время сборки образа?
Миграции нельзя выполнять при сборке образа, потому что база данных на этом этапе ещё не запущена и недоступна.
Поэтому миграции запускают в entrypoint.sh, когда контейнер уже стартовал и может подключиться к БД.
Так миграции применяются каждый раз при запуске новой версии приложения, а не только при сборке образа
5. Что произойдет, если миграции завершатся ошибкой при запуске контейнера?
Контейнер остановится и приложение не запустится
6. В чем разница между линтерами (flake8) и форматерами (black)?
Линтеры - анализируют код на ошибки, проблемы стиля, потенциальные баги
Форматеры - автоматически форматируют код (отступы, пробелы, переносы строк)
7. Как pre-commit хуки помогают в разработке?
 - Автоматическая проверка кода перед коммитом
 - Единый стиль кода в репозитории
 - Раннее обнаружение проблем (до попадания в репозиторий)
 - Экономия времени (не нужно запускать проверки вручную)
 - Защита от плохого кода (коммит блокируется при ошибках)

Ссылки

GitHub: <https://github.com/gh-u14/AppDev/>